

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

УДК 004.42

УТВЕРЖДАЮ
Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель департамента
программной инженерии
Н.А. Павлочев
«_____» _____ 2026 г.

Выпускная квалификационная работа

на тему «Андроид-приложение для совместного планирования поездок»

по направлению подготовки 09.03.04 «Программная инженерия»

Научный руководитель
Доцент департамента программной
инженерии факультета компьютерных наук
Брейман Александр Давидович

Подпись, Дата

Выполнил
студент группы БПИ223
образовательной программы
«Программная инженерия»
Новик Илья Иванович

Подпись, Дата

Москва 2026

Содержание

Реферат	3
Abstract	4
Термины, определения и сокращения	5
Введение	6
1 Предметная область и существующие решения	8
1.1 Предметная область и актуальность	8
1.2 Описание существующих решений	8
1.2.1 Splitwise	8
1.2.2 Wanderlog	8
1.2.3 TravelSpend	9
1.2.4 TripIt	9
1.2.5 Яндекс Путешествия	9
1.2.6 RUSSPASS	9
1.3 Описание разрабатываемого решения	9
1.4 Сравнительный анализ существующих решений	10
1.5 Средства коммуникации общего назначения и класс решаемой задачи . .	11
1.6 Функциональный разрыв и обоснование ниши разрабатываемого приложения	11
1.7 Выводы по главе	11
2 Проектирование системы	13
2.1 Пользовательские сценарии	13
2.2 Архитектура системы	13
2.2.1 Архитектура клиентской части	14
2.2.2 Архитектура серверной части	15
2.2.3 Взаимодействие клиента и сервера	15
2.2.4 Иллюстративные фрагменты кода	19
2.3 Модель данных и связи сущностей	20
2.4 Подсистемы и функциональное назначение	21
2.5 Выбор методов и средств реализации	22
2.5.1 Серверная часть	23
2.5.2 Хранилище данных	23
2.5.3 Клиентская часть	24
2.5.4 Интеграционные компоненты	25
2.5.5 Обоснование выбранных паттернов	25
2.6 Нефункциональные решения	26

2.7	Выводы по главе	26
3	Программная реализация системы	28
3.1	Реализация подсистемы авторизации и профиля	32
3.1.1	Реализация на клиенте	32
3.1.2	Реализация на сервере	32
3.1.3	Пользовательский поток (владелец/участник)	33
3.1.4	Результат для пользователя	33
3.2	Реализация подсистемы поездок и участников	34
3.2.1	Реализация на клиенте	34
3.2.2	Реализация на сервере	38
3.2.3	Пользовательский поток (владелец/участник)	38
3.2.4	Результат для пользователя	38
3.3	Реализация подсистемы идей, обсуждений и маршрута	39
3.3.1	Реализация на клиенте	39
3.3.2	Реализация на сервере	40
3.3.3	Пользовательский поток (владелец/участник)	40
3.3.4	Результат для пользователя	40
3.4	Реализация подсистемы расходов и балансов	41
3.4.1	Реализация на клиенте	41
3.4.2	Реализация на сервере	41
3.4.3	Пользовательский поток (владелец/участник)	42
3.4.4	Результат для пользователя	42
3.5	Реализация подсистем погоды, AI-подсказок и уведомлений	43
3.5.1	Реализация на клиенте	43
3.5.2	Реализация на сервере	44
3.5.3	Политика безопасности и релевантности AI-подсказок	44
3.5.4	Пользовательский поток (владелец/участник)	44
3.5.5	Результат для пользователя	45
3.6	Реализация офлайн-режима и синхронизации	45
3.7	Проверка сквозных пользовательских сценариев	45
3.8	Трассировка требований, реализации и проверки	48
3.9	Автоматизированное тестирование	48
3.10	Выводы по главе	49
	Заключение	50
	Список использованных источников	51

Реферат

Общий объём работы: 51 страница; иллюстраций — 11; таблиц — 7; фрагментов кода — 5; использованных источников — 24; приложений — 0.

КЛЮЧЕВЫЕ СЛОВА: Android-приложение, совместное планирование поездок, маршрут по дням, учет расходов, баланс участников, уведомления, AI-предложения.

Выпускная квалификационная работа посвящена разработке программного комплекса для совместного планирования поездок: серверной части и мобильного клиента для ОС Android. Актуальность исследования обусловлена тем, что в реальной практике групповой подготовки поездки пользователи распределяют задачи между несколькими сервисами, что приводит к дублированию данных и потере контекста решений.

Объектом исследования является процесс совместного планирования поездок в малых группах пользователей. Предметом исследования выступают пользовательские сценарии, функциональные требования и проектные решения программного комплекса, объединяющего ключевые этапы планирования в едином цифровом пространстве.

Цель работы состоит в проектировании и реализации указанного комплекса в соответствии с техническим заданием и документами приёмки. Критерий достижения цели — выполнение требований технического задания и успешная приёмка по сценариям из программы и методики испытаний. В рамках работы проведен анализ предметной области и существующих решений, сформулирован функциональный разрыв, спроектирована архитектура и разграничение зон ответственности мобильного и серверного кода, определён модульный состав, а также описана реализация.

Создан полнофункциональный клиент-серверный программный комплекс: реализованы мобильный клиент для ОС Android и серверная часть, обеспечивающие управление поездками и участниками, работу с идеями и маршрутом, учет расходов и балансов, прогноз погоды, AI-предложения, настройки уведомлений по типам событий, предусмотренным техническим заданием, а также кэширование данных поездки для просмотра без сети.

Практическая значимость работы заключается в применимости полученного результата для повседневного группового планирования поездок и в возможности дальнейшего развития системы в рабочей среде.

Abstract

Total volume: 51 pages; figures — 11; tables — 7; code fragments — 5; references — 24; appendices — 0.

KEY WORDS: Android application, collaborative trip planning, day-by-day itinerary, expense tracking, participant balance, push notifications, AI suggestions.

This graduation thesis is focused on developing a software system for collaborative trip planning: a backend service and an Android mobile client. The study is relevant because in real group planning practice users typically split their work across multiple digital services, which causes data duplication and loss of decision context.

The object of the study is the collaborative trip planning process in small user groups. The subject of the study is user scenarios, functional requirements, and design decisions for that software system combining key planning stages within a single digital workspace.

The goal is to design and implement the system in line with the technical specification and acceptance documents. Success is evidenced by meeting the requirements and passing the scenarios in the test program and methodology. The work includes domain analysis and review of existing solutions, identification of a functional gap, system architecture, separation of responsibilities between client and server code, the module structure, and description of the software implementation.

The outcome is a complete client–server software system: an Android mobile app and a backend, providing trip and participant management, ideas and itinerary planning, expense tracking and balance calculation, weather forecast, AI-based suggestions, notification settings for the event types required by the technical specification, and cached trip data for viewing while offline.

The practical significance lies in the applicability of the proposed solution to everyday collaborative trip planning and in its readiness for further system evolution in an operational environment.

Термины, определения и сокращения

В настоящей выпускной квалификационной работе применяют следующие термины с соответствующими определениями:

Активность — элемент маршрута, назначенный на конкретный день поездки.

Баланс — агрегированное представление финансовых обязательств участников поездки.

Владелец — пользователь, создавший поездку и управляющий её параметрами и доступом участников.

Идея — пользовательское предложение активности или места до включения в маршрут.

Комментарий идеи — сообщение пользователя в обсуждении конкретной идеи.

Локализация UI — отображение интерфейса на языке, выбранном политикой локализации приложения.

Офлайн-кэш — локально сохранённые данные поездки для просмотра при отсутствии сети.

Офлайн-синхронизация — согласование локальных и серверных данных после восстановления связи; полный контур отложенных правок вне сети может развиваться по мере эволюции приложения.

Поездка — сущность планирования, объединяющая даты, участников, идеи, маршрут и расходы.

Расход — запись о планируемой или уже произведенной трате в рамках поездки.

Участник — пользователь, присоединенный к поездке и участвующий в совместных действиях.

AI-предложение — автоматически сгенерированная рекомендация, которую пользователь может сохранить в список идей.

В настоящей выпускной квалификационной работе применяют следующие сокращения:

ПМИ — программа и методика испытаний.

ТЗ — техническое задание.

ТП — текст программы.

Введение

Планирование групповых поездок в повседневной практике чаще всего выполняется в нескольких цифровых каналах одновременно: обсуждения ведутся в мессенджерах, маршрут фиксируется в картографических сервисах, расходы учитываются в отдельных приложениях, а погодные данные и справочная информация ищутся во внешних источниках. Такая фрагментация приводит к дублированию действий, потере контекста принятых решений и повышенной координационной нагрузке на организатора поездки [8].

Актуальность темы обусловлена потребностью в едином мобильном инструменте, который поддерживает полный цикл совместного планирования: от формулирования идей и обсуждений до формирования маршрута по дням и прозрачного учета расходов. Для малых групп пользователей критична не только доступность отдельных функций, но и их согласованная работа в общем сценарии [7][8].

Объектом исследования является процесс совместного планирования поездок в малых группах.

Предметом исследования являются пользовательские сценарии, функциональные требования и проектные решения мобильного приложения, обеспечивающего совместную работу участников поездки в едином информационном пространстве.

Цель выпускной квалификационной работы — разработка и реализация программного комплекса для совместного планирования поездок: серверной части и мобильного клиента для ОС Android, обеспечивающих согласованные с техническим заданием сценарии совместной работы участников и снижение издержек групповой координации в рамках заявленной функциональной модели.

Критерии достижения цели. Для настоящей работы цель считается достигнутой, если реализованы требования технического задания [7], проведена приёмка по сквозным пользовательским сценариям в соответствии с программой и методикой испытаний [22], результаты зафиксированы в третьей главе настоящей работы и согласованы с текстом программы [9]. Таким образом, обоснованность цели опирается на трассируемость «требования — реализация — проверка», а не на сравнение с универсальными средствами голосовой связи или мессенджерами, относящимися к иному классу задач (см. 1.5).

Объём по клиентским платформам. Мобильный клиент в объёме ВКР реализован для ОС Android. Серверная часть опирается на HTTP API и не привязана к конкретной мобильной платформе.

Для достижения поставленной цели решаются следующие задачи:

1. Проанализировать предметную область и выявить ограничения фрагментированного подхода к планированию поездок.
2. Рассмотреть существующие решения и определить их функциональные границы в контексте совместного планирования.

3. Сформулировать функциональный разрыв и обосновать целевую нишу разрабатываемого приложения.
4. Спроектировать архитектуру системы, модель данных и ключевые пользовательские сценарии.
5. Обосновать выбор методов и средств реализации.
6. Разработать серверную часть приложения (backend), реализующую API, правила доступа и хранение данных.
7. Разработать клиентскую часть Android-приложения, реализующую ключевые пользовательские сценарии совместного планирования поездок.
8. Провести проверку сквозных пользовательских сценариев и зафиксировать результаты.

Практическая значимость работы состоит в создании прикладного Android-приложения, в котором объединены пользовательски значимые функции: управление поездкой, совместный доступ, работа с идеями и маршрутом, учет расходов и балансов, погодный контекст, AI-предложения, уведомления по поддерживаемым типам событий и кэширование данных поездки для просмотра без сети [7].

Выпускная квалификационная работа включает три главы. В первой главе рассматриваются предметная область и существующие решения, во второй - проектирование системы, в третьей - программная реализация и результаты проверки сквозных пользовательских сценариев.

1 Предметная область и существующие решения

1.1 Предметная область и актуальность

Совместное планирование поездки представляет собой последовательность взаимосвязанных действий: выбор направления и дат, формирование списка активностей, обсуждение альтернатив, распределение расходов и согласование итогового маршрута. На каждом этапе участники принимают коллективные решения, которые влияют на дальнейшие шаги. Изменение одного параметра поездки, например даты или состава участников, затрагивает сразу несколько подсистем планирования.

Ключевая проблема предметной области заключается в разрыве контекста между инструментами, когда обсуждения, маршрут и расходы фиксируются в разных сервисах. В результате участники получают несколько версий одного и того же плана, а владелец поездки вынужден вручную синхронизировать информацию. Это увеличивает вероятность ошибок, усложняет контроль договоренностей и снижает прозрачность процесса [8].

Таким образом, актуальной является задача создания единого мобильного пространства, в котором данные поездки связаны на уровне пользовательского сценария, а переход между этапами планирования выполняется без потери контекста [7][8][9].

1.2 Описание существующих решений

Сравнение существующих продуктов строится на публично доступных сведениях: официальных сайтах, публикациях вендоров и материалах магазинов приложений; ссылки на источники даны в описании каждого из сервисов ниже. Рассматриваемые сервисы закрывают отдельные аспекты группового планирования и позволяют определить функциональные границы существующих подходов.

1.2.1 Splitwise

Splitwise ориентирован на учет совместных расходов и расчет взаиморасчетов между участниками группы [1]. Сервис предоставляет удобные механизмы фиксации трат и контроля задолженности, однако не покрывает задачи обсуждения идей и формирования маршрутного плана по дням в рамках единого процесса [1][8].

1.2.2 Wanderlog

Wanderlog предназначен для планирования поездок и совместного редактирования маршрута [2]. Сильной стороной решения является маршрутная часть, однако в контексте целевой постановки разрабатываемого приложения финансовые и дискуссионные сценарии представлены ограниченно и не формируют целостный процесс принятия решений [2][8].

1.2.3 TravelSpend

TravelSpend специализируется на фиксации расходов во время поездки [3]. Продукт эффективен для финансового учета, но не позиционируется как единая среда, объединяющая идеи, обсуждения, маршрут и совместное управление доступом [3][8].

1.2.4 TripIt

TripIt решает задачу агрегации информации о поездке и автоматизации отдельных организационных операций [4]. При этом сервис не ориентирован на полноценную коллективную модерацию идей и их управляемый перенос в маршрут в рамках одного пользовательского пространства [4][8].

1.2.5 Яндекс Путешествия

В публикации от 3 апреля 2025 года для сервиса «Яндекс Путешествия» описан режим совместной поездки по ссылке-приглашению [5]. Данный функционал улучшает совместный доступ к информации, однако основной акцент сервиса остается на сценариях подбора и бронирования, а не на полном цикле группового планирования с интегрированным учетом обсуждений и расходов [5][8].

1.2.6 RUSSPASS

RUSSPASS ориентирован на подбор маршрутов и туристический контент [6]. По открытым описаниям сервис поддерживает информационные и навигационные возможности путешествия, но не заявляет полный набор механизмов коллективного планирования, включая управляемые обсуждения и баланс расходов в рамках одной рабочей области [6][8].

Проведенный обзор показывает, что существующие продукты преимущественно специализируются на отдельных задачах. Наиболее частый подход - закрытие одного функционального сегмента (маршрут, расходы или контент), а не сквозного сценария совместной подготовки поездки.

1.3 Описание разрабатываемого решения

Разрабатываемое решение направлено на объединение ключевых пользовательских действий в рамках одной поездки как центральной сущности. Функциональная модель сервиса включает:

1. Авторизацию и персональный профиль пользователя.
2. Создание и управление поездками с параметрами дат, валюты и состава участников.
3. Механизм приглашений для совместного доступа.

4. Подсистему идей и обсуждений с ролями владельца и участника.
5. Маршрут по дням с операциями добавления и переноса активностей между днями.
6. Учет расходов с балансом и статусами оплаты.
7. Прогноз погоды в городах поездки.
8. AI-подсказки с сохранением предложения в список идей.
9. Настройки уведомлений по поддерживаемым типам событий [7].
10. Кэширование данных поездки для просмотра без сети [7].

С точки зрения пользовательского результата разрабатываемое приложение формирует единую рабочую область, в которой обсуждение и фиксация решений не разрываются между внешними сервисами. Это позволяет уменьшить количество повторных действий и повысить прозрачность группового планирования [7][8].

1.4 Сравнительный анализ существующих решений

Для сопоставления решений использована матрица критериев, непосредственно связанных с целевой задачей разрабатываемого приложения.

Таблица 1.1 — Сравнение существующих решений и разрабатываемого приложения

Сервис	Совместное планирование в одном пространстве	Идеи и обсуждения	Маршрут по дням	Расходы и баланс	Погода	AI-подсказки / совместный доступ	Приглашения
Splitwise	Нет	Нет	Нет	Да	Нет	Нет	Да
Wanderlog	Нет	Нет	Да	Нет	Нет	Нет	Да
TravelSpend	Нет	Нет	Нет	Да	Нет	Нет	Нет
TripIt	Нет	Нет	Да	Нет	Нет	Нет	Нет
Яндекс Путешествия	Нет	Нет	Нет	Нет	Нет	Нет	Да
RUSSPASS	Нет	Нет	Нет	Нет	Нет	Нет	Нет
Разрабатываемое приложение	Да	Да	Да	Да	Да	Да	Да

Примечание: в матрице используются только бинарные оценки. Если функция в стороннем сервисе присутствует фрагментарно и не закрывает критерий целиком, в таблице фиксируется значение “Нет”.

Результаты сравнительного анализа показывают, что у рассмотренных решений присутствуют сильные стороны в отдельных функциональных направлениях, однако в рамках выбранных критериев и источников [1][2][3][4][5][6] не прослеживается целостное покрытие сквозного сценария «идея -> обсуждение -> маршрут -> расходы -> синхронизация» в одном пользовательском пространстве [8].

1.5 Средства коммуникации общего назначения и класс решаемой задачи

Универсальные мессенджеры и сервисы видеосвязи (в том числе чаты и программы вроде Zoom) предназначены прежде всего для обмена сообщениями и организации сеансов связи. Они не задают единой структурированной модели поездки на уровне данных: состав участников и роли, маршрут по дням, расходы и статусы оплаты в одном согласованном контуре без ручного переноса между приложениями. Поэтому прямое сопоставление таких средств с разрабатываемым приложением как «функциональных аналогов» некорректно: это разные классы инструментов.

Целевой эффект разрабатываемого решения в постановке настоящей работы — снизить фрагментацию между этапами совместной подготовки поездки (см. раздел 1.1) за счёт единого мобильного приложения и серверной части, реализующих требования [7], а не заменить голосовое общение или переписку.

1.6 Функциональный разрыв и обоснование ниши разрабатываемого приложения

По итогам анализа можно выделить функциональный разрыв, релевантный предметной области совместного планирования поездок:

1. Отсутствие единого пространства, где одновременно поддерживаются обсуждение идей, маршрутное планирование и финансовая прозрачность.
2. Недостаточная связность между этапами принятия решения и фактическим включением активности в маршрут.
3. Если поездка ведётся в разных несвязанных сервисах, без стабильного интернета согласованные сведения о ней в полном виде получить нельзя: данные в сети, в одном приложении целой картины нет.

Позиционирование разрабатываемого приложения определяется как интегрированный мобильный сервис для небольших групп пользователей, которым необходима единая среда принятия и фиксации решений по поездке. В отличие от специализированных решений, приложение ориентировано на целостный процесс, где каждая подсистема работает в контексте одной поездки и общей ролевой модели [7][8][9].

1.7 Выводы по главе

В первой главе рассмотрена предметная область совместного планирования поездок и обоснована актуальность ее цифровой интеграции. Проведен обзор существующих решений, показавший, что распространенные сервисы закрывают отдельные подзадачи, но в рамках принятых критериев сравнения не демонстрируют целостного сквозного сценария коллективной подготовки поездки.

На основе сравнительного анализа сформулирован функциональный разрыв и определено позиционирование разрабатываемого приложения как решения, объединяющего в едином пользовательском процессе управление поездкой, обсуждение идей, маршрут, расходы и вспомогательные сервисы. Полученные результаты задают основу для второй главы, в которой рассматриваются проектные решения и архитектура системы.

2 Проектирование системы

2.1 Пользовательские сценарии

Проектирование системы опирается на ролевую модель «владелец поездки - участник поездки» и на сквозные сценарии, формирующие целевую пользовательскую ценность. Владелец инициирует поездку и управляет доступом, участники участвуют в обсуждениях, планировании маршрута и распределении расходов.

Ключевые сценарии, учитываемые при проектировании, представлены в таблице.

Таблица 2.1 — Ключевые пользовательские сценарии

Сценарий	Компоненты	Результат
Создание поездки и приглашение участника	Клиент Android, API поездок, API приглашений	Формируется общая рабочая область поездки для владельца и участника
Идея → обсуждение → маршрут	Подсистема идей, новые комментарии real-time, модуль маршрута	Предложение участника после обсуждения фиксируется в плане дня
Добавление расхода и пересчет баланса	Подсистема расходов, участники поездки, клиентский расчет итогов по данным API	Пользователи видят актуальные доли и итоговый баланс
Погода и AI-предложение	Погодный модуль, AI-модуль, подсистема идей	Пользователь получает контекст и сохраняет предложение в идеи
Работа без сети и синхронизация	Локальное хранилище (кэш), API при восстановлении сети	При отсутствии сети доступны кэшированные данные поездки для просмотра; после появления связи состояние обновляется с сервера

Для демонстрации связности модулей в главе 2 используется базовый поток «идея -> обсуждение -> маршрут», поскольку именно он отражает переход от коллективного обсуждения к формальному плану поездки. Поток реализуется как последовательность проверяемых шагов: участник формирует идею, участники обсуждают ее в комментариях, затем по результату обсуждения идея либо переносится в маршрут по дням, либо остается в списке идей для последующей доработки.

2.2 Архитектура системы

Система проектируется как клиент-серверное приложение. Клиентская часть реализует пользовательские экраны, локальное хранение и координацию сценариев. Сер-

верная часть централизует бизнес-правила, доступ к данным и интеграцию с внешними сервисами. Такой подход поддерживает единый источник актуального состояния поездки при сохранении отзывчивости интерфейса [12][13][14].

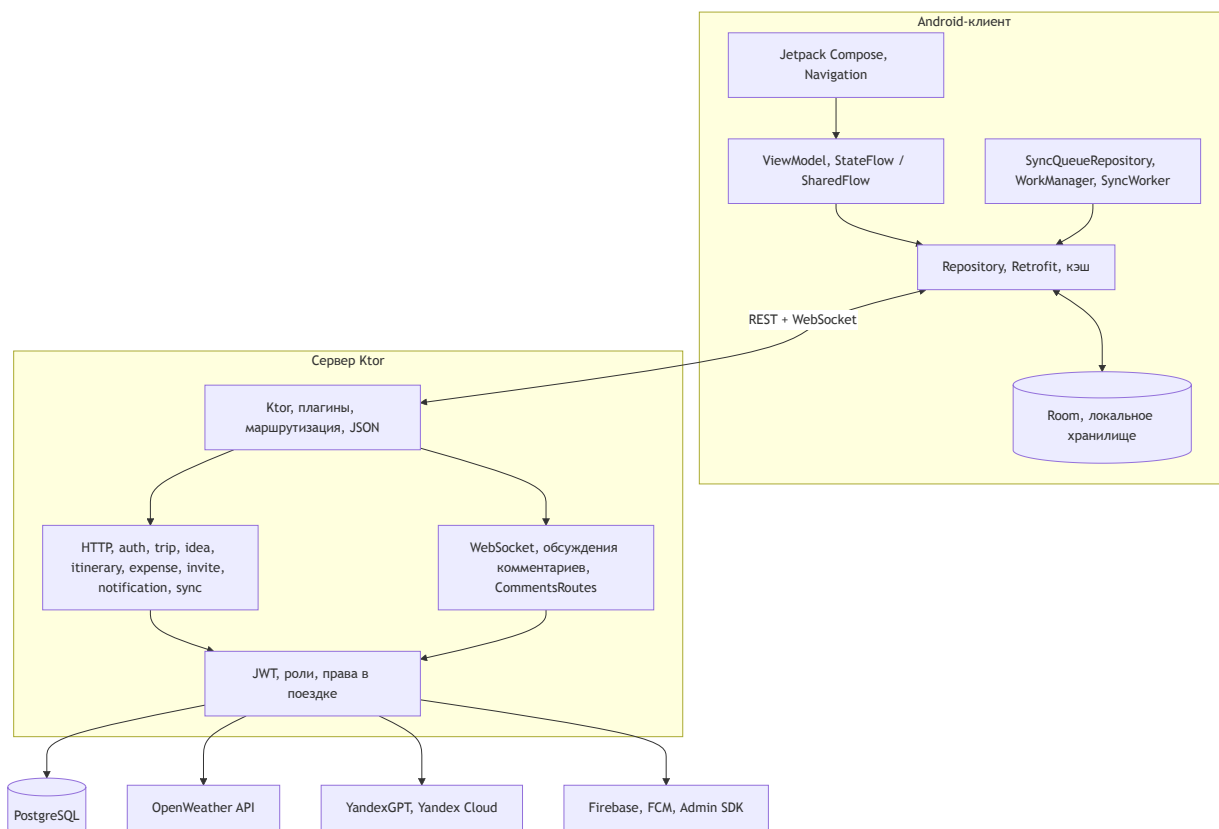


Рисунок 2.1 — Обзорная архитектура системы: Android клиент -> Backend -> PostgreSQL/интеграции.

Схема отражает работу в сети (запросы к API, обновления в реальном времени) и вне сети (хранение копии на устройстве с последующим обновлением). Это соответствует сценарию, где подключение не гарантировано [19].

2.2.1 Архитектура клиентской части

Клиент построен по многоуровневой схеме UI->ViewModel->Repository->Datasource.

1. Слой интерфейса включает экраны Jetpack Compose и навигацию между пользовательскими сценариями.
2. В слое представления используются ViewModel и потоки состояния (StateFlow / SharedFlow), что задает цикл «событие -> изменение состояния -> обновление экрана».
3. Репозитории инкапсулируют работу с REST API и локальным кэшем, чтобы экран не зависел от источника данных напрямую.

4. Инфраструктурный слой включает внедрение зависимостей (Hilt) и фоновые задачи (WorkManager) для обновления данных после появления сети.

Такое разделение снижает связанность экранов с сетевыми деталями и позволяет развивать клиентские сценарии по модулям (поездки, идеи, маршрут, расходы) без дублирования сетевой и кэш-логики [14][15][16][19].

2.2.2 Архитектура серверной части

Сервер реализован как модульный backend на Ktor с явным разделением ответственности:

1. Точка входа приложения инициализирует конфигурацию, подключение к базе данных, аутентификацию и общие плагины (логирование, сериализация, обработка ошибок, WebSocket).
2. HTTP-маршруты сгруппированы по предметным областям (`auth`, `trip`, `idea`, `itinerary`, `expense`, `invite`, `notification`, `sync`), что упрощает сопровождение API.
3. Предметные проверки и доступ к данным сосредоточены в серверных модулях и репозиториях, чтобы правила валидации и доступа не дублировались по маршрутам.
4. Интеграции с внешними сервисами (погода, AI, уведомления) вынесены в отдельные компоненты, изолированные от ядра предметной логики.

Для обсуждений используется отдельный WebSocket-канал: публикация нового комментария выполняется через событие в канале, при этом проверка прав участия в поездке остается на сервере [12][13][17][18].

2.2.3 Взаимодействие клиента и сервера

Взаимодействие строится в двух режимах:

1. синхронный режим: REST API для операций по поездкам, идеям, маршруту, расходам и настройкам;
2. событийный режим: WebSocket для новых комментариев и уведомления для фоновых оповещений.

Сервер выступает источником истинного состояния, а клиент поддерживает локальную копию данных для отображения и кэширования. При нестабильной сети сохраняется доступ к ранее загруженной информации; после восстановления связи выполняется обновление с сервера. Расширение до полноценной отложенной отправки правок, выполненных без подключения, относится к перспективе развития [12][13][19].

Сводный перечень REST-эндпоинтов, реализованных в серверных модулях (префикс `/v1` и JSON в теле запросов/ответов), приведен в таблице 2.2. Схема полей, специфичных для DTO, определяется в коде проекта; в таблице фиксированы метод, путь и назначение.

Таблица 2.2 — Основные маршруты HTTP API (префикс `/v1`, JSON в теле запросов/ответов)

Метод	Путь	Назначение (кратко)
POST	<code>/v1/auth/google</code>	Вход через Google ID token; выдача пары <code>access/refresh</code>
POST	<code>/v1/auth/dev</code>	Разработческий вход; выдача пары <code>access/refresh</code>
POST	<code>/v1/auth/refresh</code>	Обновление пары <code>access/refresh</code> по <code>refreshToken</code> в теле
POST	<code>/v1/auth/logout</code>	Завершение сессии (требуется <code>Authorization: Bearer access</code>)
GET	<code>/v1/users/me</code>	Профиль текущего пользователя
PATCH	<code>/v1/users/me</code>	Обновление профиля
DELETE	<code>/v1/users/me</code>	Удаление учетной записи (предусмотренный сценарий)
POST	<code>/v1/trips</code>	Создание поездки
GET	<code>/v1/trips</code>	Список поездок текущего пользователя
GET	<code>/v1/trips/{tripId}</code>	Детали поездки (по идентификатору)
PATCH	<code>/v1/trips/{tripId}</code>	Изменение атрибутов поездки
DELETE	<code>/v1/trips/{tripId}</code>	Удаление поездки
POST	<code>/v1/trips/{tripId}/archive</code>	Архивирование поездки
POST	<code>/v1/trips/{tripId}/transfer-owner</code>	Передача роли владельца
POST	<code>/v1/trips/{tripId}/join</code>	Присоединение по внутреннему сценарию
GET	<code>/v1/trips/{tripId}/members</code>	Состав участников поездки
DELETE	<code>/v1/trips/{tripId}/members/{memberId}</code>	Удаление участника (с проверкой прав)

Продолжение табл. 2.2		
Метод	Путь	Назначение (кратко)
POST	/v1/trips/{tripId}/ invite	Создание ссылки-приглашения; ответ: токен и URL
GET	/v1/invites/{token}	Публичные сведения о поездке по токenu приглашения
POST	/v1/invites/{token}/ accept	Принятие приглашения (аутентификация)
GET	/v1/trips/{tripId}/ ideas	Список идей по поездке
POST	/v1/trips/{tripId}/ ideas	Создание идеи
GET	/v1/ideas/{ideaId}	Просмотр идеи
PATCH	/v1/ideas/{ideaId}	Редактирование идеи
DELETE	/v1/ideas/{ideaId}	Удаление идеи
POST	/v1/ideas/{ideaId}/ approve	Согласование/утверждение идеи
POST	/v1/ideas/{ideaId}/ reject	Отклонение идеи
POST	/v1/ideas/{ideaId}/ convert-to-activity	Перенос идеи в активности маршрута
GET	/v1/ideas/{ideaId}/ comments	Список комментариев (пагинация опциональна)
DELETE	/v1/comments/ {commentId}	Удаление комментария (по правам)
GET	/v1/trips/{tripId}/ cities/search	Подсказки по городам
GET	/v1/trips/{tripId}/ places/search	Подсказки по местам
GET	/v1/trips/{tripId}/ itinerary	Маршрут: дни и активности
PATCH	/v1/itinerary/days/ {dayId}	Редактирование дня (в т.ч. город/координаты)
POST	/v1/itinerary/days/ {dayId}/activities	Добавление активности в день
PATCH	/v1/itinerary/ activities/ {activityId}	Редактирование активности
DELETE	/v1/itinerary/ activities/ {activityId}	Удаление активности

Продолжение табл. 2.2		
Метод	Путь	Назначение (кратко)
POST	/v1/itinerary/ activities/ {activityId}/move	Перенос активности (в т.ч. между днями)
POST	/v1/itinerary/days/ {dayId}/activities/ reorder	Порядок активностей в дне
POST	/v1/trips/{tripId}/ itinerary/trim-out- of-range	Подрезка маршрута по диапа- зону дат поездки
GET	/v1/trips/{tripId}/ expenses	Список расходов по поездке
POST	/v1/trips/{tripId}/ expenses	Создание расхода
GET	/v1/expenses/ {expenseId}	Просмотр расхода
PATCH	/v1/expenses/ {expenseId}	Редактирование расхода и сплитов
DELETE	/v1/expenses/ {expenseId}	Удаление расхода
POST	/v1/trips/{tripId}/ai/ suggestions	Генерация AI-подсказок (по- литика и фильтрация на сер- вере)
POST	/v1/ai/suggestions/ {id}/save-to-ideas	Сохранение предложения в идеи поездки
GET	/v1/trips/{tripId}/ weather	Прогноз погоды для контекста поездки
POST	/v1/trips/{tripId}/ weather/refresh	Форс-обновление погоды с внешнего API
GET	/v1/notifications	Лента уведомлений
PATCH	/v1/notifications/ {id}/read	Пометка уведомления прочи- танным
POST	/v1/notifications/ read-bulk	Массовая отметка прочитан- ным
POST	/v1/push-tokens	Регистрация FCM/Push токе- на устройства
DELETE	/v1/push-tokens/ {token}	Снятие токена
GET	/v1/users/me/ notification-settings	Настройки уведомлений

Продолжение табл. 2.2		
Метод	Путь	Назначение (кратко)
PATCH	/v1/users/me/ notification-settings	Изменение настроек уведомлений
GET	/v1/sync/changes	Синхронизация: выборка изменений (сервер)
POST	/v1/sync/changes	Синхронизация: пакет отложенных команд с клиента
POST	/v1/uploads/images	Загрузка изображения (медиа)

В коде сервера часть путей для приглашений дублируется без префикса `/v1` (`/trips/{tripId}/invite` и т. п.) в целях обратной совместимости. Новые комментарии в обсуждении идут, кроме REST, через WebSocket-канал (см. выше).

2.2.4 Иллюстративные фрагменты кода

Сервер (Ktor, Kotlin). Перечисленные маршруты регистрируются в модульных Route-функциях. Фрагмент кода 2.1 демонстрирует обмен refresh-токена: тело `RefreshRequest`, обращение к `AuthTokenService`, ответ `RefreshResponse`; типичная ветка ошибки — ответ 401 Unauthorized.

Листинг 2.1: Фрагмент маршрута `/v1/auth/refresh` (сервер)

```
post("/v1/auth/refresh") {
    val request = call.receive<RefreshRequest>()
    val tokens = try {
        AuthTokenService.refreshTokens(request.refreshToken)
    } catch (ex: AuthFlowException) {
        call.respond(
            HttpStatusCode.Unauthorized,
            mapOf("error" to mapOf("code" to ex.code, "message" to ex.message))
        )
        return@post
    }
    call.respond(RefreshResponse(
        accessToken = tokens.accessToken,
        refreshToken = tokens.refreshToken
    ))
}
```

Мобильный клиент (OkHttp, Kotlin). Срабатывание `Authenticator` при сетевом отказе, связанном с аутентификацией, и обновление пары `access/refresh` показаны на

фрагменте кода 2.2 (см. также описание токенов в главах 2–3). Сетевой вызов — через `authRefreshApi`, отдельно от основного клиента с `AuthInterceptor`.

Листинг 2.2: `SessionAuthenticator` (середина `authenticate`): повтор с новым `access`, затем `refresh`

```
synchronized(lock) {
    val latestAccessToken = sessionStore.getAccessToken()
    if (!latestAccessToken.isNullOrBlank() && latestAccessToken != requestToken) {
        return response.request.newBuilder()
            .header("Authorization", "Bearer " + latestAccessToken)
            .build()
    }
    val refreshToken = sessionStore.getRefreshToken()
    if (refreshToken.isNullOrBlank()) {
        sessionCleaner.clearSessionBlocking()
        return null
    }
    val refreshResponse = runCatching {
        authRefreshApi.refresh(RefreshRequest(refreshToken)).execute()
    }.onFailure { }.getOrNull() ?: return null
    if (!refreshResponse.isSuccessful) { return null }
    val refreshedTokens = refreshResponse.body() ?: return null
    sessionStore.setTokens(
        accessToken = refreshedTokens.accessToken,
        refreshToken = refreshedTokens.refreshToken
    )
    return response.request.newBuilder()
        .header("Authorization", "Bearer " + refreshedTokens.accessToken)
        .build()
}
```

Проверка кодов, лог *AppLogger* и полный *TERMINAL_AUTH* опущены для компактности; ветка отображает источник *SessionAuthenticator.kt*.

2.3 Модель данных и связи сущностей

Концептуальная модель данных строится вокруг сущности поездки как корневого объекта. Все функциональные подсистемы связаны с поездкой и наследуют ее контекст (даты, участники, ограничения, валюту).

Основные связи сущностей описываются следующим образом:

1. Пользователь может участвовать в нескольких поездках, а одна поездка содержит нескольких участников.

2. Идея принадлежит конкретной поездке и может быть преобразована в активность маршрута.
3. Комментарии являются элементами обсуждения идеи и связаны с автором.
4. Маршрут представлен набором дней, каждый день содержит упорядоченный список активностей.
5. Расход относится к поездке и включает распределение долей между выбранными участниками.
6. Погодные данные привязываются к выбранному городу и датам поездки.
7. Предложение AI создается в контексте поездки и может быть сохранено как идея.

Для серверного хранения применяется реляционная модель данных на основе PostgreSQL. В текущей реализации структура таблиц инициализируется схемой проекта и поддерживается в кодовой базе. На стороне клиента сохраняется локальное состояние, необходимое для восстановления пользовательского процесса при временной потере сети [10][11][19].

2.4 Подсистемы и функциональное назначение

Для проектирования и последующей реализации выделены функциональные подсистемы, каждая из которых решает отдельную задачу, но работает в рамках общего сценария поездки.

Таблица 2.3 — Подсистемы приложения и их назначение

Подсистема	Функции	Пользовательская ценность
Авторизация и профиль	Вход, управление профилем, выход, удаление аккаунта	Контролируемый доступ к персональным данным
Поездки и участ-ники	Создание/редактирование поездки, управление участниками, приглашения	Быстрое формирование совместного пространства
Идеи и обсужде-ния	Создание идей, комментарии, модерация статуса	Прозрачная фиксация коллективных решений
Маршрут по дням	Добавление, перенос и упорядочивание активностей	Формирование исполнимого плана поездки
Расходы и ба-ланс	Добавление расходов, разделение между участниками, контроль статусов оплаты	Прозрачность финансовой ответственности
Погода и AI-подсказки	Прогноз по городу поездки, генерация и сохранение предложений	Поддержка принятия решений по активности
Настройки уве-домлений	Управление категори-ями уведомлений	Снижение риска пропуска важных изменений
Офлайн и син-хронизация	Локальное кэширова-ние данных поездки; обновление с сервера при восстановлении се-ти	Просмотр ранее загруженных данных без подключения

Представленное разделение на подсистемы соответствует требованиям ТЗ и задает основу для модульной реализации, где пользователь наблюдает целостный процесс, а не набор изолированных экранов [7].

2.5 Выбор методов и средств реализации

Выбор методов и средств реализации выполнен по явным критериям: (1) поддержка мобильного сценария и офлайн-работы, (2) предсказуемость сопровождения, (3) совместимость со стекком Android/Backend, (4) стоимость доработок при расширении продукта.

2.5.1 Серверная часть

Серверная часть ориентирована на централизованную реализацию предметных правил и внешних интеграций. Выбран стек Kotlin + Ktor [12][13] по следующим причинам:

1. единый язык с клиентом снижает затраты на обучение и сопровождение команды;
2. Ktor предоставляет REST и WebSocket в одном фреймворке, что покрывает сценарии API и обсуждений;
3. стек допускает модульную структуру маршрутов и тестирование без сложной дополнительной инфраструктуры.

В качестве альтернатив рассматривались Java/Spring и Node.js/Express; для данного проекта выбран Kotlin/Ktor как более совместимый со стеком Android и требованиями текущего проекта.

Архитектурно на сервере принят паттерн «маршрут -> предметный модуль -> репозиторий». Такой подход выбран для того, чтобы валидация входных данных, проверка ролей и операции с базой данных проходили через единые точки принятия решений.

2.5.2 Хранилище данных

Предметные данные (поездки, участники, идеи, маршрут, расходы) хранятся в PostgreSQL [10][11]. Причины, по которым вместо встраивания базы в один локальный файл или хранения предметных данных в виде набора JSON-документов выбрана реляционная СУБД:

1. Модель предметной области включает многочисленные отношения: расходы относятся к конкретной поездке, в расходе участвуют выбранные участники, дни и активности маршрута иерархически связаны с поездкой. Схему, заданную такими отношениями, при проектировании обычно отображают в виде набора взаимосвязанных таблиц; в данном проекте реляционный стиль хранения выбран как согласующийся с перечисленной структурой.
2. Сценарий нередко обновляет сразу несколько согласованных строк. Реляционная СУБД оформляет пакет шагов транзакцией (свойства ACID), благодаря чему при сбое не остаётся частично применённого изменения и снижаются конфликты при параллельных сеансах — это важно при совместном планировании.
3. Описание структуры БД (какие таблицы и поля) закреплено в репозитории и применяется при развёртывании, поэтому одинаковую пустую схему проще неоднократно поднять, например, в тестовой среде и в рабочей, без ручной копии структуры.

Альтернативой служат встраиваемые (файловые) СУБД и документоориентированные СУБД, в которых сущности часто объединяют в документы, структурно близкие к JSON. При той же сети отношений, если взаимосвязанные сущности поездки лежат в отдельных документах, существенную часть согласованного обновления нередко пришлось бы дублировать в прикладном слое, тогда как в PostgreSQL эти отношения естественно задавать таблицами, ссылками и транзакциями средствами СУБД, что в рамках настоящей работы снижает риск дублирования правил целостности в коде приложения.

2.5.3 Клиентская часть

Указанные выше критерии в мобильном клиенте детализируются так. Критерий (1) (сеть и офлайн) обеспечивается разделением сетевого и локального путей в репозиториях, локальным снимком данных и фоновыми заданиями (WorkManager) [19]. Критерий (2) (сопровождение) согласуется с выбором библиотек из Android Jetpack и опирается на публикуемую вендором документацию [14][15][16][19]. Критерий (3) (согласованность со стеком backend) закреплён в использовании Kotlin [13] в паре с сервером и в типизированном доступе к HTTP API. Критерий (4) (доработки) согласуется с устойчивой к изменениям разбивкой на UI, ViewModel, репозитории, сетевой и кэш-слой.

Мобильный клиент реализуется для Android на Kotlin [13] с Jetpack Compose, Dagger Hilt, Retrofit, OkHttp и WorkManager [14][15][16][19]. Для локального кэширования вместо прямых SQL-вызовов в прикладном коде используется Room для табличного хранения сущностей снимка и Jetpack DataStore (Preferences) для пар ключ–значение, что снижает риск ошибок при смене схемы и согласуется с рекомендациями по персистентности в Android [14].

Пользовательский уровень: вместо экранов на View/XML (без Jetpack Compose) [14] взят декларативный UI на Jetpack Compose: при динамических списках и согласовании с ViewModel легче избегать расхождения разметки и кода, чем при сопоставлении XML-разметки с состоянием. Внедрение зависимостей: Dagger Hilt [15] выбран, чтобы сократить ручной код по сравнению с базовым Dagger, где модули и внедрение на компоненты Android приходилось бы собирать и связывать явно.

Сетевой уровень: для REST-вызовов используется Retrofit вместе с OkHttp [16] как распространённый в среде Android вариант объявления API в виде интерфейсов. WorkManager [19] задействован для фоновой работы при обрывистой сети.

Архитектура: MVVM+Repository+DI. Состояние экрана сосредоточено в ViewModel, репозитории скрывают разделение сети и локального кэша, Hilt [15] предоставляет зависимости по модулям, что снижает зависимость слоя интерфейса от деталей сетевого обмена и локального кэширования.

2.5.4 Интеграционные компоненты

Для расширения пользовательского процесса в проекте используются внешние сервисы прогноза погоды и AI-генерации. В подсистеме офлайн-работы применяется механизм фоновой обработки задач синхронизации. Такой выбор позволяет сохранить работоспособность ключевых сценариев даже при частичной недоступности внешних источников [17][18][19].

Для интеграций используется адаптерный подход: вызовы внешних API инкапсулируются в отдельных компонентах, а прикладные модули работают с внутренними моделями. Это уменьшает объем изменений при замене провайдера и снижает риск каскадных правок в доменной логике.

Поставщиками по конкретным контурам выбраны следующие решения.

OpenWeather API [17] используется для прогноза погоды по выбранному городу в датах поездки. По публичной документации поставщика реализуемы устойчивые сценарии краткосрочного прогноза, достаточные для подсистемы «Погода и AI-подсказки» без введения отдельного погодного движка.

Для генерации текстовых идей в контексте поездки задействована модель YandexGPT в Yandex Cloud [18]. Это согласуется с вариантом облачного развертывания сервера (см. ниже) и снижает сопутствующую сложность интеграции в выбранной облачной среде.

Для доставки push-уведомлений и серверной отправки сообщений на Android применяются *Firebase Cloud Messaging* и *Firebase Admin SDK* [23][24]. В экосистеме Android это стандартный, документированный путь, что снижает риск отказа при сопровождении типовых сценариев токенов и фоновой доставки.

2.5.5 Обоснование выбранных паттернов

Обоснование архитектурных паттернов в проекте сводится к проверяемым требованиям эксплуатации:

1. сценарии выполняются в нестабильной сети, поэтому требуется отдельная обработка онлайн- и офлайн-путей (локальный кэш и фоновое обновление при появлении сети);
2. в системе много экранов и форм, поэтому состояние интерфейса должно быть централизовано в *ViewModel*, а не распределено по компонентам UI;
3. предметные ограничения (роли, принадлежность к поездке, валидность приглашения) должны проверяться на сервере в единых точках, чтобы избежать расхождений между сценариями;
4. внешние сервисы могут быть частично недоступны или заменяться, поэтому интеграции изолируются адаптерным слоем;

5. модульная декомпозиция по подсистемам уменьшает стоимость локальных изменений и снижает риск регрессий при доработках.

2.6 Нефункциональные решения

Нефункциональные решения сформулированы с позиции эксплуатационной устойчивости и качества пользовательского опыта.

1. Согласованность данных. Сервер обеспечивает единые правила валидации и хранения, а также разрешение коллизий при синхронизации, чтобы все участники поездки видели согласованное состояние после завершения пользовательских операций над данными поездки [12][13].
2. Надежность пользовательских сценариев. Критические действия (идеи, маршрут, расходы) проектируются с учетом обработки ошибок и восстановления после сетевых сбоев [7][19].
3. Безопасность доступа. В пользовательских сценариях доступ к действиям внутри поездки ограничен аутентификацией и участием пользователя в поездке; ролевые операции владельца и участника разграничены на уровне API-маршрутов модерации и управления доступом [7][12].
4. Интеграционная устойчивость. При недоступности отдельных внешних сервисов базовые функции совместного планирования остаются доступными, а вспомогательные данные и уведомления дорабатываются после восстановления интеграций [17][18][23][24].
5. Эксплуатационная применимость. Архитектура допускает развертывание серверной части в облачной среде и публикацию мобильного клиента в стандартном канале распространения Android-приложений [20][21].

Мобильный клиент опирается на локальный кэш как на снимок данных на устройстве. Пока сеть доступна, для отображения запрашиваются сведения с сервера (кэш в этом режиме не подменяет сервер); ответы при этом дублируются в кэш. При пропадании сети пользователю показывают данные из кэша, то есть те, что были сохранены ранее при соединении. Сценарий накопления и последующей двунаправленной синхронизации с сервером вне объема диплома не детализируется.

2.7 Выводы по главе

Во второй главе выполнено проектирование системы на уровне пользовательских сценариев, архитектуры, модели данных и разделения на подсистемы. Показано, что выбранные проектные решения ориентированы на поддержку полного цикла совместного планирования поездки в едином мобильном приложении.

Сформированы проектные основания для реализации ключевых модулей: авторизации, управления поездкой и участниками, идей и обсуждений, маршрута, расходов, погодного и AI-контекста, уведомлений, офлайн-работы и синхронизации. Обоснован выбор методов и средств реализации, достаточный для перехода к этапу программной реализации, представленному в следующей главе.

3 Программная реализация системы

В данной главе представлено описание программной реализации системы и итогового пользовательского интерфейса. По структуре изложения глава построена по модульному принципу: для каждой функциональной подсистемы зафиксировано, что реализовано на клиенте, что реализовано на сервере, как выглядит пользовательский поток и какой результат получает пользователь. Пользовательская Android-сборка публикуется в RuStore: <https://www.rustore.ru/catalog/app/nvk.cotrip>.

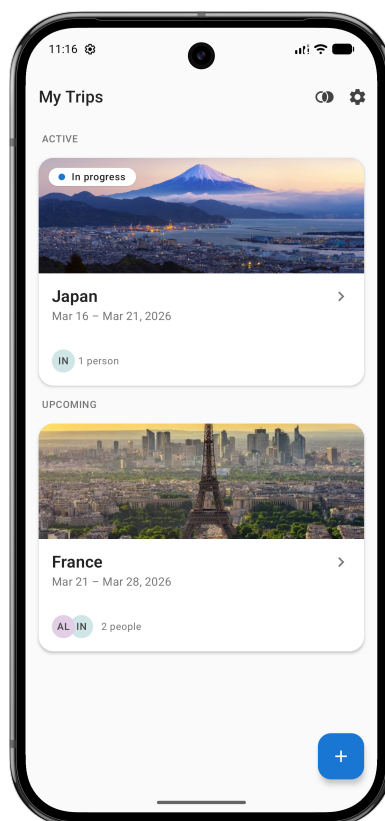


Рисунок 3.1 — Интерфейс приложения: экран «Список поездок».

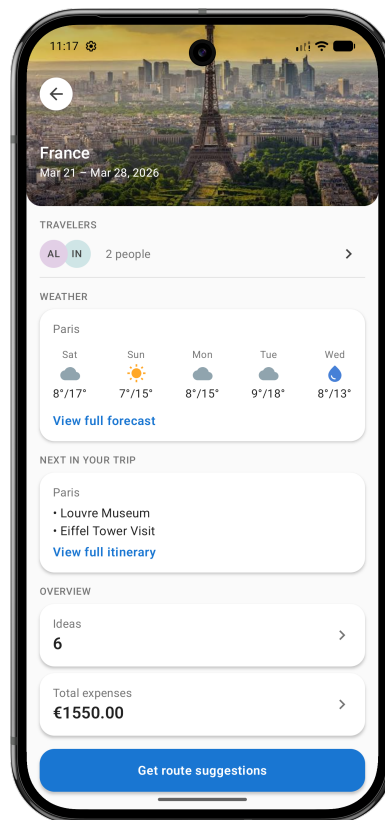


Рисунок 3.2 — Интерфейс приложения: экран «Детали поездки».

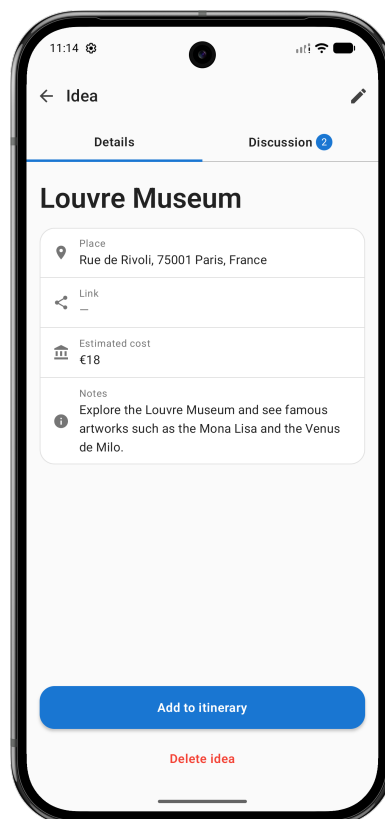


Рисунок 3.3 — Интерфейс приложения: экран «Идея».

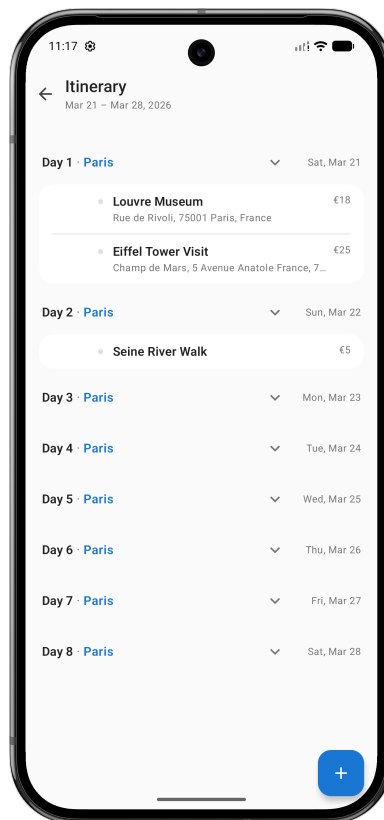


Рисунок 3.4 — Интерфейс приложения: экран «Маршрут».

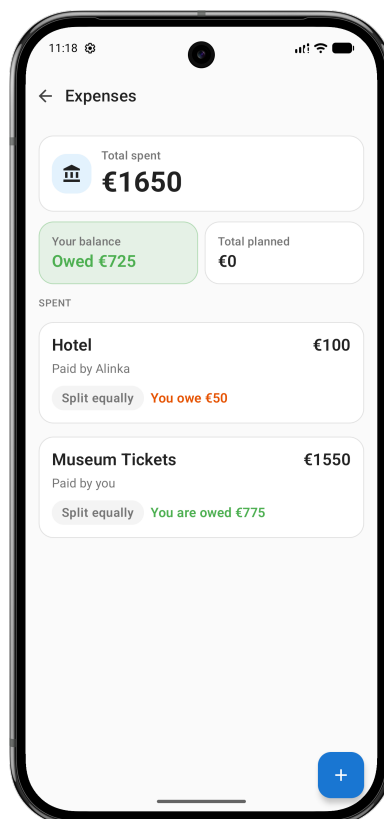


Рисунок 3.5 — Интерфейс приложения: экран «Расходы».

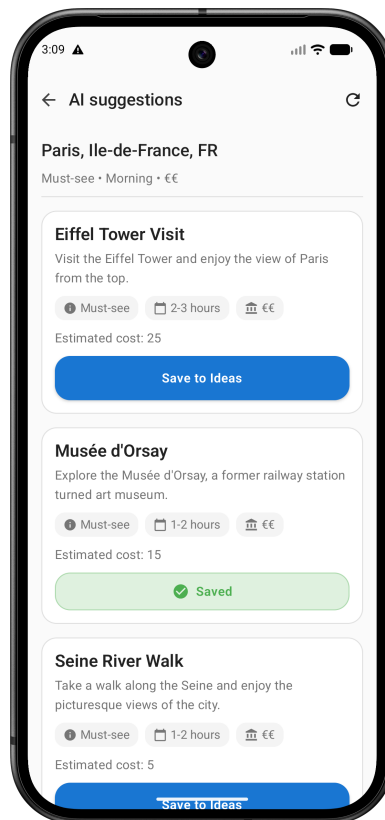


Рисунок 3.6 — Интерфейс приложения: экран «ИИ-подсказки».

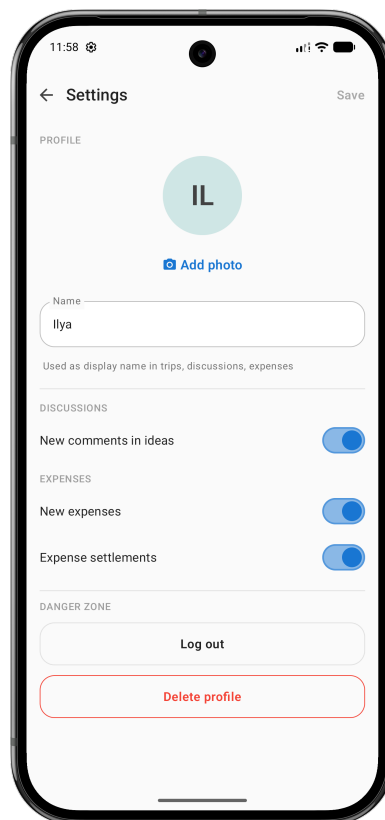


Рисунок 3.7 — Интерфейс приложения: экран «Настройки».

В качестве демонстрационного сценария рассматривается совместное планирование поездки, где владелец создает поездку, приглашает участника, формирует идеи

и маршрут, распределяет расходы, использует погодные данные и AI-подсказки. Такой выбор позволяет проверить не отдельные экраны, а целостность системы и связность реализованных модулей [7][8][9].

3.1 Реализация подсистемы авторизации и профиля

3.1.1 Реализация на клиенте

На стороне Android реализован экран входа и связанный поток инициализации сессии. Пользователь проходит авторизацию и после успешного входа попадает в рабочий режим приложения. В клиентской части также реализован экран настроек, где доступны изменение имени, фотография профиля, завершение сессии и удаление учетной записи в предусмотренном пользовательском сценарии [7][9][14]. В интерфейсе применена политика локализации: при системной локали `ru` приложение отображает русский UI, в остальных локалях — английский UI.

Визуально подсистема построена так, чтобы отделять аутентификационный этап от этапа рабочей навигации. До входа пользователю доступа только авторизация; после входа активируется доступ к поездкам и остальным модулям. Такой переход упрощает модель доступа и снижает риск ситуации, когда неавторизованный пользователь может открыть внутренние экраны приложения.

После успешного входа `access`-токен и `refresh`-токен записываются в локальное защищённое хранилище. К исходящим запросам к API при наличии `access`-токена в заголовке `Authorization: Bearer` добавляется его значение. Механизм `Authenticator` клиентской библиотеки `OkHttp` срабатывает, когда требуется пройти аутентификационный вызов: выполняется отдельный запрос `POST /v1/auth/refresh` с `refresh`-токеном в теле, в хранилище записывается обновлённая пара `access/refresh`; после этого исходный запрос к API повторяется с новым `access`-токеном. Если `refresh` неуспешен, сессия сбрасывается, и снова доступен сценарий входа.

3.1.2 Реализация на сервере

На серверной стороне реализованы маршруты аутентификации и сервисы выдачи токенов, включая проверку входных учетных данных и управление жизненным циклом сессии. Сервер отвечает за валидацию входа и разграничение доступа к защищенным маршрутам, связанным с поездками и пользовательскими данными [12][13].

Сервер выдаёт `access`-токен (JWT) ограниченного срока действия и `refresh`-токен с более длительным сроком действия, согласованный в паре с `access`; по маршруту `/v1/auth/refresh` при валидном `refresh` возвращается обновлённая пара `access/refresh`. Сроки задаются конфигурацией бэкенда. В варианте настроек по умолчанию `access` действует 15 мин, срок сессии, в рамках которой валиден `refresh`, — 30 сут, при успешном обмене на `/v1/auth/refresh` срок сессии продлевается от момента операции. Для операций профиля используются защищенные маршруты, требующие валидного контекста

пользователя.

3.1.3 Пользовательский поток (владелец/участник)

1. Пользователь выполняет вход в приложении.
2. Система проверяет учетные данные и создает активную сессию.
3. Пользователь открывает настройки и изменяет профильные данные.
4. Обновленные данные сохраняются и отражаются в интерфейсе.

3.1.4 Результат для пользователя

Пользователь получает контролируемый доступ к данным поездки, персонализированный профиль и понятный сценарий управления учетной записью. На уровне взаимодействия это снижает риск ошибок доступа и повышает доверие к системе как к личному рабочему пространству.

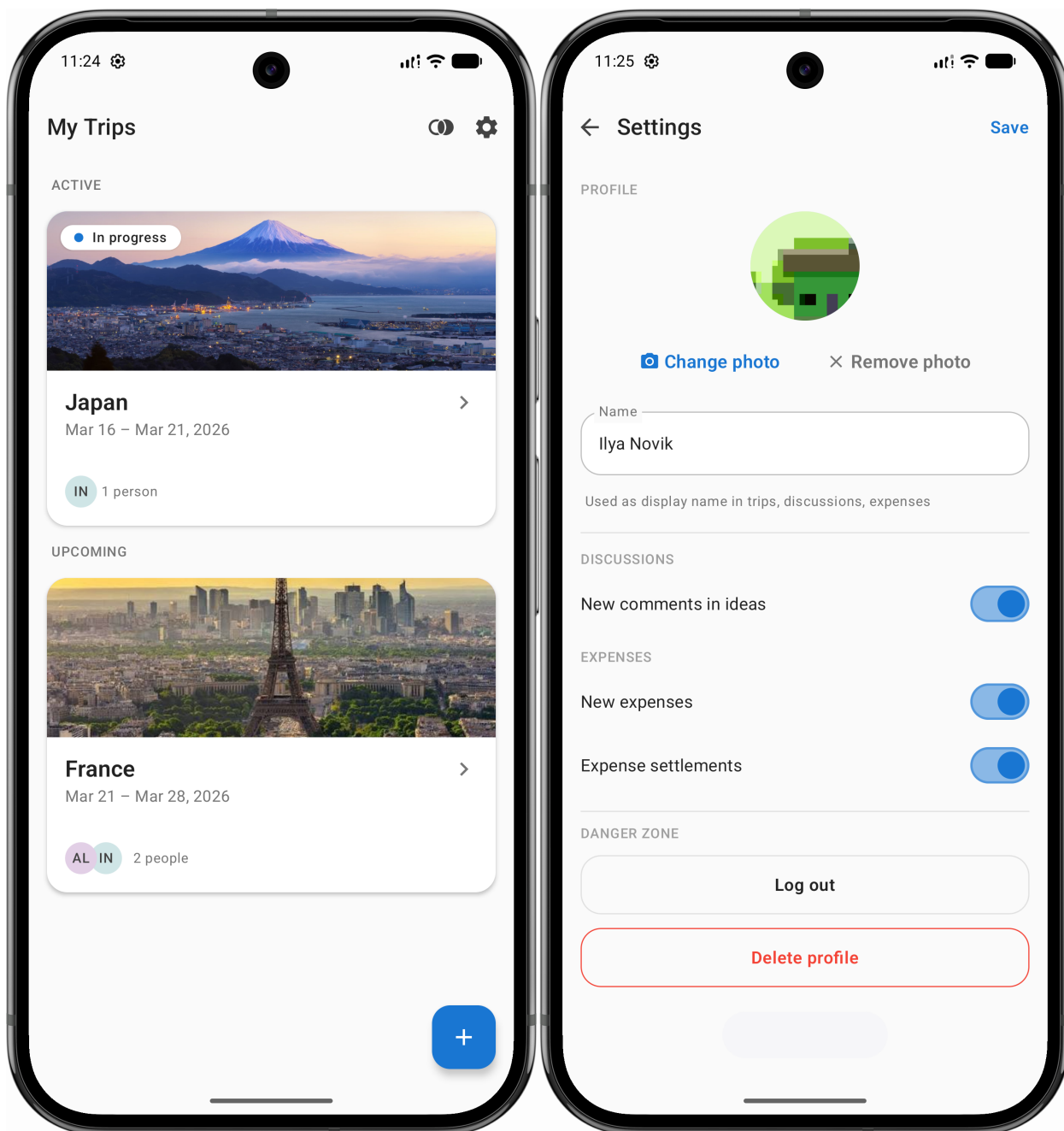


Рисунок 3.8 — Авторизация и профиль: экран успешного входа и экран настроек после изменения данных.

3.2 Реализация подсистемы поездок и участников

3.2.1 Реализация на клиенте

Клиентская часть включает экраны списка поездок, формы создания и редактирования поездки, карточку поездки и экран участников. В формах реализованы пользовательские валидаторы дат, названия и базовых параметров поездки, включая валюту и описание. В интерфейсе владельца доступны действия по приглашению участников и управлению составом поездки [7][9].

Отдельное внимание уделено представлению поездки как центра контекста. Пользователь сначала работает со списком поездок, затем открывает детали кон-

кретной поездки и уже из нее переходит в подсистемы идей, маршрута, расходов и уведомлений. Такая навигация уменьшает фрагментацию и делает переходы между задачами последовательными.

Ниже приведены сокращённые фрагменты реализации сценария *список поездок* (репозиторий `cotrip`): сетевое обновление и кэш, модель экрана и оболочка Compose, аналогично стилю фрагментов кода в главе 2. Полные исходники: `TripRepositoryImpl.kt`, `TripsListViewModel.kt`, `TripsListScreen.kt`.

Репозиторий (Retrofit, кэш). Фрагмент кода 3.1 — интерфейс `TripRepository` (фрагмент) и реализация `refreshTrips`: пагинация `listTrips` по курсору, запись в `TripsCacheStore` внутри `safeLocalMutation` (см. `RepositoryLocalOps.kt` в репозитории `cotrip`).

Листинг 3.1: `TripRepository` (фрагмент) и `TripRepositoryImpl.refreshTrips` (сокр.)

```
interface TripRepository {
    val trips: Flow<List<TripDto>>
    suspend fun refreshTrips(): Result<Unit>
}

class TripRepositoryImpl @Inject constructor(
    private val api: CoTripApi,
    private val tripsCacheStore: TripsCacheStore,
) : TripRepository {
    override val trips: Flow<List<TripDto>> = tripsCacheStore.trips
    override suspend fun refreshTrips(): Result<Unit> {
        return try {
            val tripList = mutableListOf<TripDto>()
            var cursor: String? = null
            do {
                val page = api.listTrips(limit = 100, cursor = cursor)
                tripList += page.items
                cursor = page.nextCursor
            } while (cursor != null)
            safeLocalMutation("refreshTrips.setTrips") {
                tripsCacheStore.setTrips(tripList)
            }
            Result.success(Unit)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}
```

Поле `val trips: Flow` в репозитории; зависимости `TripRepositoryImpl` сокращены.

Модель представления списка. Фрагмент кода 3.2 — `TripsListViewModel`: `Hilt`, `StateFlow/SharedFlow` для состояния и `Toast`, `onEvent` (навигация и обновления). Тело `observeTrips` / `observeTripMembers` (агрегирование карточек) опущено.

Листинг 3.2: `TripsListViewModel` (сокр.: события и обновление данных)

```
@HiltViewModel
class TripsListViewModel @Inject constructor(
    @ApplicationContext private val appContext: Context,
    private val appNavigator: AppNavigator,
    private val tripRepository: TripRepository,
    private val syncPullRepository: SyncPullRepository,
) : ViewModel() {
    private val _state =
        MutableStateFlow<TripsListUiState>(TripsListUiState.Loading)
    val state: StateFlow<TripsListUiState> = _state.asStateFlow()
    private val _effects = MutableSharedFlow<TripsListEffect>(extraBufferCapacity
        = 1)
    val effects: SharedFlow<TripsListEffect> = _effects.asSharedFlow()
    init {
        observeTrips()
        observeTripMembers()
        refreshTrips(isUserRefresh = false)
    }
    fun onEvent(event: TripsListEvent) {
        when (event) {
            TripsListEvent.OnSettingsClick ->
                appNavigator.navigate(Destination.Settings)
            TripsListEvent.OnCreateTripClick ->
                appNavigator.navigate(Destination.CreateTrip)
            TripsListEvent.OnJoinTripClick ->
                appNavigator.navigate(Destination.JoinTrip())
            TripsListEvent.OnAutoRefresh -> refreshTrips(isUserRefresh = false)
            TripsListEvent.OnUserRefresh -> refreshTrips(isUserRefresh = true)
            is TripsListEvent.OnTripClick ->
                appNavigator.navigate(Destination.TripDetails(event.id))
            TripsListEvent.OnTogglePast -> Unit
        }
    }
    private fun observeTrips() {}
    private fun refreshTrips(isUserRefresh: Boolean) {
        viewModelScope.launch {
            tripRepository.refreshTrips()
            syncPullRepository.pull()
        }
    }
}
```

```

    }
  }
}

```

В исходнике *isRefreshing*, начальная загрузка *Loading*, *Toast* при сбоях, полный *refreshTrips* см. *TripsListViewModel.kt*.

Экран списка (Jetpack Compose). Фрагмент кода 3.3 — *TripsListScreen*: *hiltViewModel*, *collectAsState* для состояния, *LaunchedEffect* для *effects* (тосты), *Scaffold* с *TopAppBar* и *FAB*. Разметка списков и *pullToRefresh* опущены.

Листинг 3.3: *TripsListScreen* (сокр.: эффекты, каркас экрана)

```

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TripsListScreen(
    viewModel: TripsListViewModel = hiltViewModel(),
) {
    val state by viewModel.state.collectAsState()
    val context = LocalContext.current
    val scrollBehavior = TopAppBarDefaults.pinnedScrollBehavior()
    val lazyColumnState = rememberLazyListState()
    LaunchedEffect(viewModel) {
        viewModel.effects.collectLatest { effect ->
            when (effect) {
                is TripsListEffect.ShowToast ->
                    Toast.makeText(context, effect.message, Toast.LENGTH_SHORT).show()
            }
        }
    }
    Scaffold(
        modifier = Modifier
            .fillMaxSize()
            .nestedScroll(scrollBehavior.nestedScrollConnection),
        topBar = {
            TopAppBar(
                title = { Text("Trips") },
                scrollBehavior = scrollBehavior,
            )
        },
        floatingActionButton = { }
    ) { _ ->
        Box(Modifier) {}
    }
}

```

Заголовок и `DisposableEffect(ON_RESUME)` в источнике используют строковые ресурсы и `OnAutoRefresh`.

3.2.2 Реализация на сервере

Сервер реализует маршруты создания, изменения, удаления и выдачи поездок, а также маршруты управления участниками и приглашениями. Для приглашений предусмотрены операции генерации и обработки ссылок, а также проверки валидности приглашения при присоединении. Действия с доступом участников выполняются с учетом роли пользователя в поездке [12][13].

Серверная часть обеспечивает согласованность состава участников: один и тот же участник не дублируется в рамках одной поездки, а невалидные приглашения не переводят пользователя в состояние «частично присоединен».

3.2.3 Пользовательский поток (владелец/участник)

1. Владелец создает поездку и задает параметры.
2. Владелец формирует приглашение и отправляет ссылку.
3. Участник открывает ссылку и присоединяется к поездке.
4. Оба пользователя видят общую поездку и единый список участников.

3.2.4 Результат для пользователя

Группа получает единое рабочее пространство поездки и прозрачный механизм совместного доступа. Не требуется вручную собирать состав в сторонних каналах: по присоединении участники оказываются в едином контуре поездки с общим набором сведений.

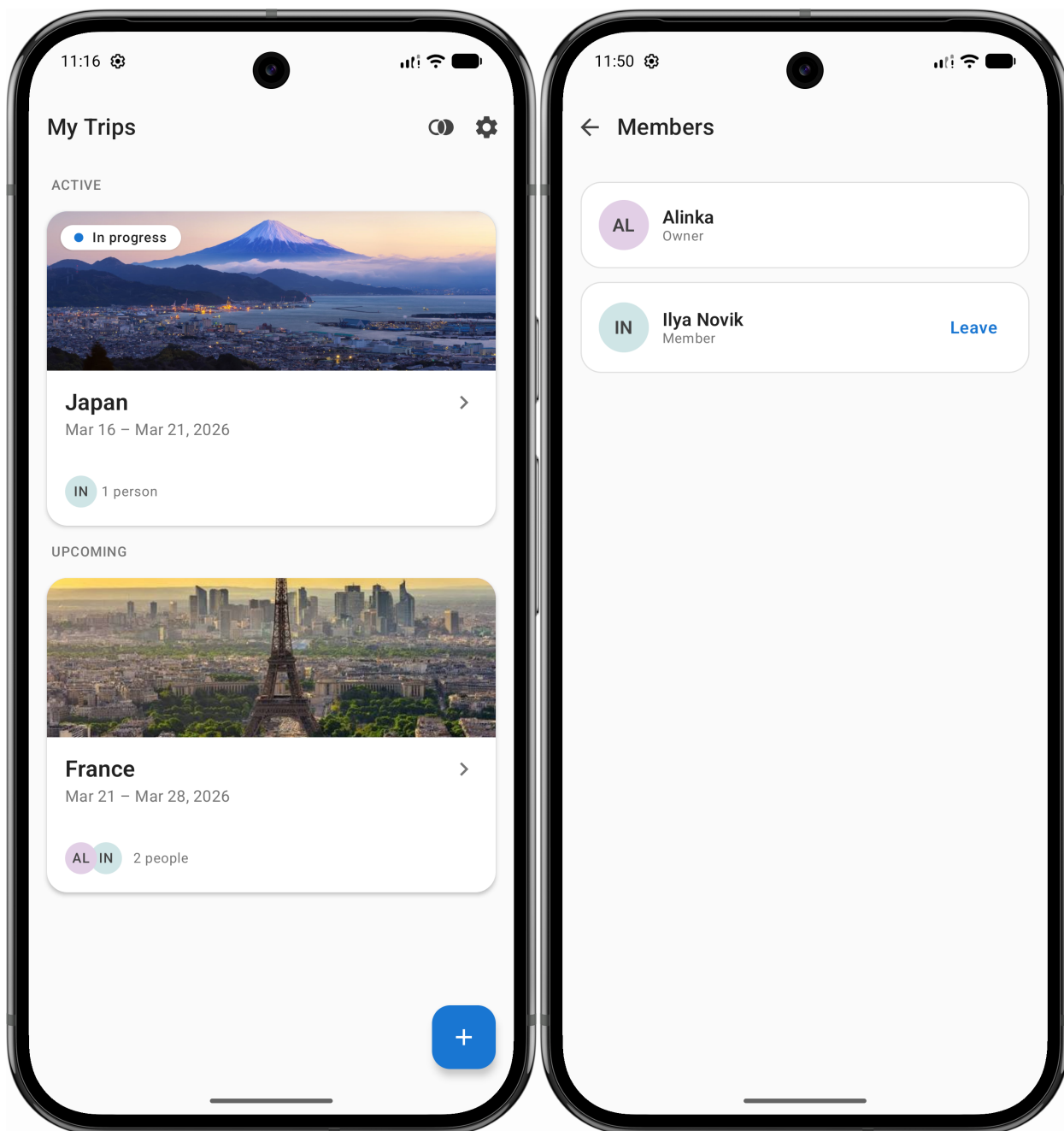


Рисунок 3.9 — Поездки и участники: создание поездки и состав участников после присоединения по ссылке.

3.3 Реализация подсистемы идей, обсуждений и маршрута

3.3.1 Реализация на клиенте

На клиенте реализованы экраны списка идей, формы создания/редактирования, карточка идеи с обсуждением, а также экран маршрута по дням. Для обсуждений используется интерфейс, поддерживающий поступление новых сообщений в реальном времени, что критично для совместного принятия решения. Для маршрута реализованы добавление активностей, изменение порядка, перенос между днями и сценарии обработки дней, выходящих за актуальный диапазон дат поездки [7][9].

Ключевой особенностью интерфейса является связка «идея -> решение -> вклю-

чение в маршрут». Пользователь не теряет контекст между обсуждением и планом: карточка идеи может быть преобразована в элемент маршрута и дальше участвует в ежедневном планировании поездки.

3.3.2 Реализация на сервере

Серверная часть включает маршруты для идей, комментариев и маршрута, а также канал real-time обновлений для обсуждений. Сервер обеспечивает хранение статусов идеи, валидацию операций модерации и согласованную запись изменений в маршрут. В текущей версии real-time канал гарантированно публикует события создания комментариев; удаление комментариев выполняется через отдельный API-маршрут с проверкой прав автора [12][13].

Такое разделение позволяет сохранить единое правило: проверка предметных ограничений выполняется на сервере, а клиент отвечает за удобство выполнения действий.

3.3.3 Пользовательский поток (владелец/участник)

1. Участник создает идею и публикует ее в поездке.
2. Участники обсуждают идею в комментариях.
3. Владелец принимает решение по идее (одобрение/отклонение).
4. Одобренная идея переносится в конкретный день маршрута.
5. Порядок активностей корректируется в соответствии с планом группы.

3.3.4 Результат для пользователя

Пользователь получает непрерывный сценарий от обсуждения к исполнимому маршруту. Это устраняет типичную проблему разрыва между «чатом с предложениями» и «финальным планом», которая характерна для разрозненных инструментов [7][8].

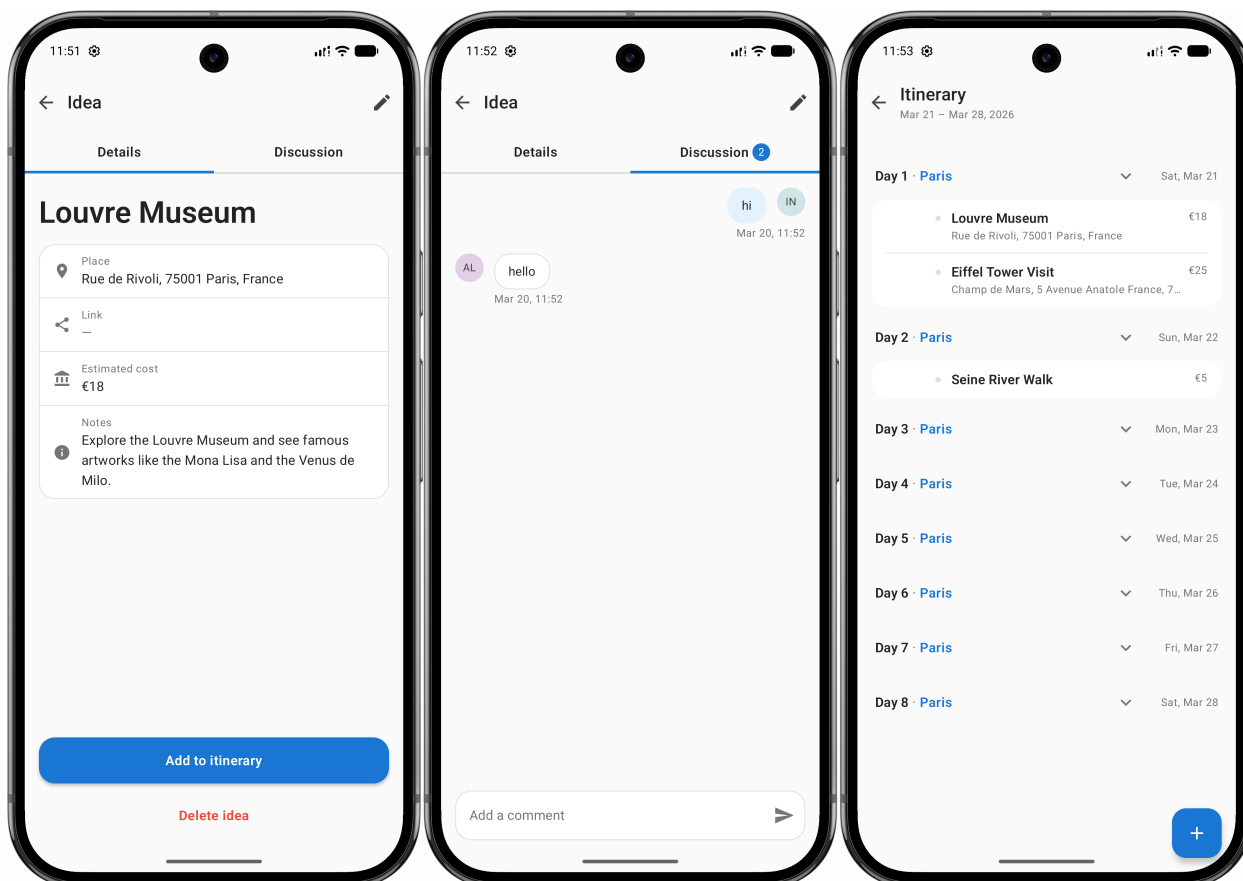


Рисунок 3.10 — Идеи, обсуждения и маршрут: сквозной поток от списка идей к обсуждению и добавлению в маршрут.

3.4 Реализация подсистемы расходов и балансов

3.4.1 Реализация на клиенте

Клиентская часть включает список расходов, форму создания/редактирования расхода, экран детализации и представление итогового баланса. Поддерживаются сценарии планируемого и оплаченного расхода, выбор плательщика, распределение долей и отметка факта оплаты участниками. Интерфейс ориентирован на то, чтобы пользователь видел не только отдельные записи, но и суммарный финансовый результат по поездке [7][9].

За счет прозрачного отображения долей и статусов оплаты снижается число уточняющих коммуникаций между участниками и упрощается закрытие финансовых обязательств по завершении поездки.

3.4.2 Реализация на сервере

Сервер реализует маршруты создания, изменения, удаления и получения расходов. В серверной логике выполняются проверки валидности суммы, участников и связки расхода с конкретной поездкой. Итоговые показатели для отображения баланса вычисляются на клиенте по данным расходов и долей участников, полученных через API [12][13].

3.4.3 Пользовательский поток (владелец/участник)

1. Пользователь создает расход с указанием суммы и состава участников.
2. Система рассчитывает доли и обновляет общую финансовую картину.
3. Участники отмечают оплату в рамках своих обязательств.
4. Баланс пересчитывается и отражается в интерфейсе.

3.4.4 Результат для пользователя

Группа получает прозрачную и проверяемую финансовую модель поездки. Пользователь видит, кто оплатил, кто должен и как меняется итоговый баланс при каждом действии.

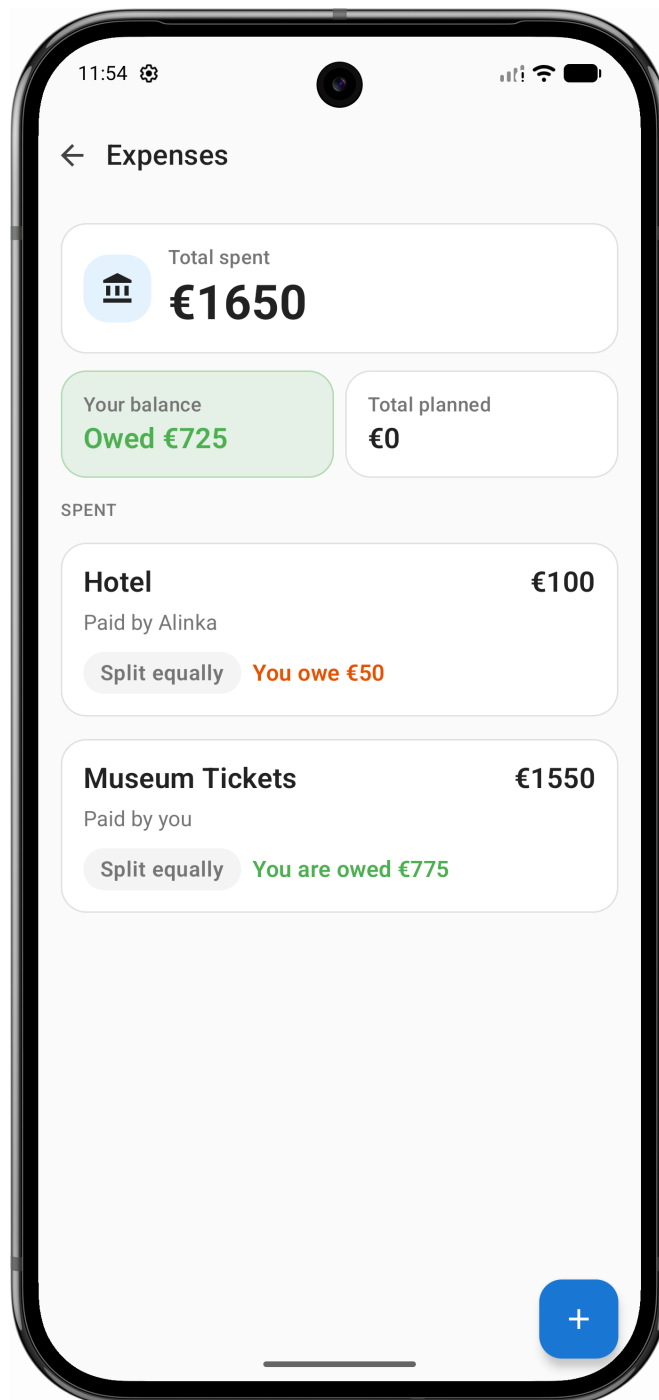


Рисунок 3.11 — Расходы и баланс: итоговые суммы и распределение долей участников.

3.5 Реализация подсистем погоды, AI-подсказок и уведомлений

3.5.1 Реализация на клиенте

На клиенте реализованы экран прогноза и погодные блоки в контексте поездки, интерфейс генерации AI-подсказок и экран настроек уведомлений. Пользователь может обновлять погодные данные, выбирать релевантный контекст поездки для AI-предложений и сохранять подходящие варианты в список идей. Рядом с результатами отображается краткое предупреждение, что ИИ может ошибаться, и важно перепроверить детали до сохранения. В настройках доступны переключатели категорий уведом-

лений [7][9].

Такая интеграция делает вспомогательные сервисы частью рабочего процесса, а не внешним шагом «после планирования».

3.5.2 Реализация на сервере

Серверная часть интегрируется с погодным и AI-провайдерами; для уведомлений — с Firebase (FCM, Admin SDK) при выделенных маршрутах регистрации токенов и отправки событий [17][18][23][24]. Погодные, AI-данные и доставка push не подменяют ядро поездки, а расширяют контекст [20].

3.5.3 Политика безопасности и релевантности AI-подсказок

Генерация идёт через Yandex Cloud [18]. Перед обращением к этапу с подсказками ввод нормализуется: по словарю в коде отсекается заведомо неприемлемый смысл (незаконные и опасные темы и др.); при срабатывании на клиент возвращается ошибка на этапе `request` с категорией `illegal_goods` или `harmful` (как ориентир, в лексиконе есть, например, `phishing` / фишинг в `illegal_goods` и подстрока `убийств` в `harmful`).

Классификация релевантности — не отдельный модуль с обученной моделью в репозитории: на сервере делается ещё один вызов к той же Yandex GPT, ответ приходит в строгом JSON: категория `travel` либо `unclear` либо `off_topic` с доверием 0–1; при `off_topic` с доверием не ниже 0,8 основная генерация не вызывается, на клиент возвращается код нарушения на этапе `request`. Так смыслово дополняется слой по словарю, не исчерпывая варианты сигнатурами.

Список подсказок из основного вызова, по-прежнему в фиксированной JSON-схеме, затем постфильтр — доразбор на сервере программным кодом, не нейросетью: снова проверяется текст, согласованность с городом (маршрутом) и с выбранным типом, временем суток, бюджетом, убираются варианты, помечаемые в коде, в том числе `off_scope`. Если список опустеет, клиенту возвращается ошибка. Напоминание на экране, что ответы вероятностные, согласовано с абзацем о клиенте: цены, часы и детали могут не совпадать с реальностью, окончательная проверка — за пользователем [7][9].

3.5.4 Пользовательский поток (владелец/участник)

1. Пользователь открывает погодный модуль поездки и обновляет прогноз.
2. Пользователь запрашивает AI-предложения для маршрута.
3. Подходящее предложение сохраняется в идеи поездки.
4. При включенных категориях уведомлений, выданном системном разрешении и доступном интернете пользователь получает уведомления о поддерживаемых со-

бытиях: новый расход, изменение статуса оплаты расхода, новый комментарий к идее [7].

3.5.5 Результат для пользователя

Пользователь быстрее принимает решения по маршруту и при включенных уведомлениях, выданном разрешении и доступе к интернету получает актуальные события поездки без постоянной ручной проверки каждого экрана.

3.6 Реализация офлайн-режима и синхронизации

В пользовательском контуре реализовано кэширование данных поездки: при отсутствии сети сохраняется возможность просмотра ранее загруженной информации, что соответствует требованию к офлайн-кэшированию в ТЗ [7].

3.7 Проверка сквозных пользовательских сценариев

В рамках верификации результата проведена функциональная проверка сквозных сценариев, отражающих ядро пользовательской ценности приложения. Проверка ориентирована на связность модулей и достижение пользовательского результата, а не только на изолированную работоспособность отдельных экранов.

Текущее покрытие модулей по реализованным возможностям представлено в таблице.

Таблица 3.1 — Покрытие модулей приложения

Модуль	Реализованные возможности	Клиент/Сервер	Статус
Авторизация и профиль	Вход, изменение профиля, завершение сессии, удаление аккаунта в пользовательском потоке	Android, Backend	Реализовано
Поездки и участники	Создание, редактирование и удаление поездки; приглашение и присоединение	Android, Backend	Реализовано
Идеи и обсуждения	Работа со списком идей, статусы, новые комментарии в реальном времени	Android, Backend, WS	Реализовано
Маршрут	Планирование по дням, перенос и переупорядочивание активностей	Android, Backend	Реализовано
Расходы и баланс	Ведение расходов, сплиты, баланс и статусы оплаты	Android, Backend	Реализовано
Погода, AI, уведомления	Прогноз, генерация AI-предложений, настройки уведомлений	Android, Backend, внешние сервисы	Реализовано
Офлайн и синхронизация	Кэширование данных поездки для просмотра без сети	Android, Backend	Реализовано

Результаты проверки сквозных сценариев приведены в таблице ниже.

Таблица 3.2 — Проверка сквозных пользовательских сценариев

Сценарий проверки	Шаги	Ожидаемый результат	Фактический результат
Создание поездки и приглашение участника	Владелец создает поездку, отправляет приглашение, участник присоединяется	У обоих пользователей отображается одна и та же поездка и общий состав участников	Сценарий воспроизводится, критических отклонений не зафиксировано
Идея → обсуждение → маршрут	Участник создает идею, новые комментарии поступают в реальном времени, идея переносится в маршрут	Идея из обсуждения становится элементом маршрута выбранного дня	Сценарий воспроизводится; результат подтвержден в рамках проведенной проверки
Управление расходом и балансом	Создается расход, задаются доли, фиксируется оплата	Баланс пересчитывается на клиенте по данным расходов и долей участников	Сценарий воспроизводится; несоответствий не обнаружено
Погода + AI-подсказка	Пользователь обновляет прогноз, генерирует AI-предложение, сохраняет в идеи	Предложение появляется в идеях, погодные данные отображаются по контексту поездки	Сценарий воспроизводится при доступности внешних сервисов
Настройки уведомлений	Пользователь изменяет категории уведомлений в настройках	При включенных категориях, выданном системном разрешении и доступном интернете система применяет выбранные категории (новый расход; изменение статуса оплаты расхода; новый комментарий к идее)	Сценарий воспроизводится; применяются только подерживаемые категории

Проверка выполнялась как ручной сценарный прогон в приложении по набору тестовых сценариев. Представленные результаты отражают выполнение ключевых сценариев и не охватывают все возможные комбинации входных данных [7][8][9].

Проведенная проверка показывает, что реализованная система закрывает целе-

вой набор пользовательских сценариев, сформулированный в ТЗ и обоснованный в аналитической части работы [7][8].

3.8 Трассировка требований, реализации и проверки

Для доказательной связи «требование — реализация — проверка» ниже приведена сводная таблица: фрагменты постановки [7] сопоставлены с модулями реализации (см. таблицу покрытия модулей выше) и с результатами сквозной проверки (таблица сценариев в настоящем разделе); приёмочные испытания по полному набору сценариев описаны в программе и методике испытаний [22].

Таблица 3.3 — Сводная трассировка требований и проверок

Фрагмент постановки (ТЗ)	Реализация (подсистемы)	Проверка
Поездки, участники, приглашения	Клиент Android, API поездок и приглашений, backend	Сквозной сценарий создания поездки и приглашения; ПМИ [22]
Идеи, обсуждения, статусы	Клиент, API идей, WebSocket комментариев, backend	Сценарий «идея → обсуждение → маршрут»; ПМИ [22]
Маршрут по дням	Клиент, API маршрута, backend	Тот же сквозной сценарий; таблица модулей; ПМИ [22]
Расходы, баланс, статусы оплаты	Клиент, API расходов, backend	Сценарий управления расходом и балансом; ПМИ [22]
Погода, AI-предложения	Клиент, интеграции, backend	Сценарий погоды и AI; ПМИ [22]
Уведомления (типы событий ТЗ)	Клиент, доставка уведомлений, backend	Сценарий настроек уведомлений; ПМИ [22]
Офлайн-кэширование данных	Клиент (кэш), backend	Подраздел об офлайн-режиме; сценарии ПМИ при отключённой сети [22]

3.9 Автоматизированное тестирование

Помимо описанной выше ручной проверки по сценариям, в проекте используются модульные и интерфейсные тесты клиента и серверной части; в конфигурации сборки заданы пороги покрытия кода (JaCoCo) как вспомогательный инженерный контроль [9]. Такие проверки дополняют верификацию пользовательского результата, но не подменяют приёмку по ПМИ. Конкретные команды и отчёты остаются на уровне репозитория и при необходимости могут запускаться локально или в контуре непрерывной интеграции.

3.10 Выводы по главе

В третьей главе представлена программная реализация приложения в модульной форме, соответствующей структуре и стилю рассмотренных примеров ВКР. Для каждой подсистемы зафиксированы реализованные возможности на клиенте и сервере, а также итоговый пользовательский результат.

Показано, что ключевая ценность системы формируется на уровне сквозных сценариев: участники группы могут совместно пройти путь от создания поездки и обсуждения идей до построения маршрута и распределения расходов. Реализация вспомогательных модулей (погода, AI, уведомления) встроена в общий процесс планирования и усиливает его практическую применимость; доступ к данным при кратковременной потере сети поддерживается за счёт кэша.

Результаты функциональной проверки по набору сценариев указывают на соответствие текущей версии приложения целевому сценарию совместного планирования, заявленному в предыдущих главах. Таким образом, после аналитического и проектного этапов в главах 1 и 2, в главе 3 получено практическое подтверждение применимости разработанной системы по результатам проведенной проверки.

Заключение

В выпускной квалификационной работе рассмотрена предметная область совместного планирования поездок и обоснована необходимость интеграции ключевых пользовательских сценариев в едином программном комплексе (серверная часть и мобильный клиент для ОС Android).

В первой главе выполнен анализ существующих решений и выявлен функциональный разрыв между специализированными сервисами и потребностью пользователей в сквозном процессе планирования; отдельно зафиксировано отличие предметной модели приложения от универсальных средств коммуникации общего назначения (см. 1.5). Во второй главе сформированы проектные решения для системы: пользовательские сценарии, архитектурная модель, разделение на подсистемы, модель данных и нефункциональные требования. В третьей главе описана программная реализация модулей системы, приведена трассировка требований к результатам проверки (см. 3.8) и зафиксированы результаты сквозных сценариев.

С позиции инженерной выпускной квалификационной работы цель — разработка и реализация программного комплекса в соответствии с техническим заданием [7] и приёмкой по программе и методике испытаний [22] — считается достигнутой: реализованы заявленные пользовательские сценарии, результаты проверки согласованы с текстом программы [9]. Полученные результаты подтверждают, что текущая версия клиента для Android совместно с серверной частью обеспечивает целевой пользовательский процесс: управление поездкой и участниками, работу с идеями и маршрутом, учет расходов и баланса, использование погодного и AI-контекста, настройку уведомлений по поддерживаемым типам событий, а также кэширование данных поездки для просмотра при отсутствии сети.

Сборка мобильного клиента публикуется в RuStore; ссылка на карточку в каталоге приведена в начале раздела 3, что согласуется с вынесением результата за рамки локальной разработки.

Границы темы по клиентским платформам: в объёме ВКР реализован мобильный клиент для Android; серверное API пригодно для подключения иных клиентов, в том числе для iOS, в рамках отдельных проектов.

Дальнейшее развитие работы связано с завершением офлайн-сценария в пользовательском контуре, развитием продукта по обратной связи пользователей и усилением эксплуатационной устойчивости при росте нагрузки.

Список использованных источников

- [1] Splitwise. Официальный сайт [Электронный ресурс]. — URL: <https://www.splitwise.com/> (дата обращения: 15.03.2026).
- [2] Wanderlog. Официальный сайт [Электронный ресурс]. — URL: <https://wanderlog.com/> (дата обращения: 15.03.2026).
- [3] TravelSpend. Официальный сайт [Электронный ресурс]. — URL: <https://travelspend.com/> (дата обращения: 15.03.2026).
- [4] TripIt. Официальный сайт [Электронный ресурс]. — URL: <https://www.tripit.com/> (дата обращения: 15.03.2026).
- [5] Яндекс. Яндекс Путешествия упростили совместные поездки, 03.04.2025 [Электронный ресурс]. — URL: <https://yandex.ru/company/news/01-03-04-2025> (дата обращения: 15.03.2026).
- [6] RUSSPASS. Карточка в Google Play [Электронный ресурс]. — URL: <https://play.google.com/store/apps/details?id=ru.russpass.tourist> (дата обращения: 15.03.2026).
- [7] Техническое задание. [Текстовый документ в составе материалов к выпускной квалификационной работе.].
- [8] Проектное предложение. [Текстовый документ в составе материалов к выпускной квалификационной работе.].
- [9] Текст программы. [Текстовый документ в составе материалов к выпускной квалификационной работе.].
- [10] PostgreSQL Global Development Group. PostgreSQL Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 15.03.2026).
- [11] Инициализационная схема базы данных проекта. [Файл схемы в составе материалов к выпускной квалификационной работе.].
- [12] Ktor. Ktor Documentation [Электронный ресурс]. — URL: <https://ktor.io/docs/welcome.html> (дата обращения: 15.03.2026).
- [13] JetBrains. Kotlin Documentation [Электронный ресурс]. — URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 15.03.2026).
- [14] Google Android Developers. Jetpack Compose [Электронный ресурс]. — URL: <https://developer.android.com/compose> (дата обращения: 15.03.2026).
- [15] Google Android Developers. Dependency Injection with Hilt [Электронный ресурс]. — URL: <https://developer.android.com/training/dependency-injection/hilt-android> (дата обращения: 15.03.2026).
- [16] Square, Inc. Retrofit [Электронный ресурс]. — URL: <https://square.github.io/retrofit/> (дата обращения: 15.03.2026).
- [17] OpenWeather. Weather API: документация [Электронный ресурс]. — URL: <https://openweathermap.org/api> (дата обращения: 15.03.2026).
- [18] Yandex Cloud. YandexGPT: Foundation Models, документация [Электронный ресурс]. — URL: <https://yandex.cloud/en/docs/ai-studio/concepts/generation/>

`models` (дата обращения: 15.03.2026).

[19] Google Android Developers. WorkManager, документация [Электронный ресурс]. — URL: <https://developer.android.com/topic/libraries/architecture/workmanager> (дата обращения: 15.03.2026).

[20] Yandex Cloud. Compute Cloud, документация [Электронный ресурс]. — URL: <https://yandex.cloud/en/docs/compute/> (дата обращения: 15.03.2026).

[21] Google Android Developers. Distribute to Google Play [Электронный ресурс]. — URL: <https://developer.android.com/distribute/google-play> (дата обращения: 15.03.2026).

[22] Программа и методика испытаний. [Текстовый документ в составе материалов к выпускной квалификационной работе.].

[23] Google. Firebase Cloud Messaging, документация [Электронный ресурс]. — URL: <https://firebase.google.com/docs/cloud-messaging> (дата обращения: 15.03.2026).

[24] Google. Set up a Firebase project and the Admin SDK, документация [Электронный ресурс]. — URL: <https://firebase.google.com/docs/admin/setup> (дата обращения: 15.03.2026).